

V02 - Scrum

Softwaretechnik: Systeme und Projekte

Sommersemester 2026

Rico Saßen (rico.sassen@lba.hs-bremen.de)

Noah Raven (noah.raven@lba.hs-bremen.de)

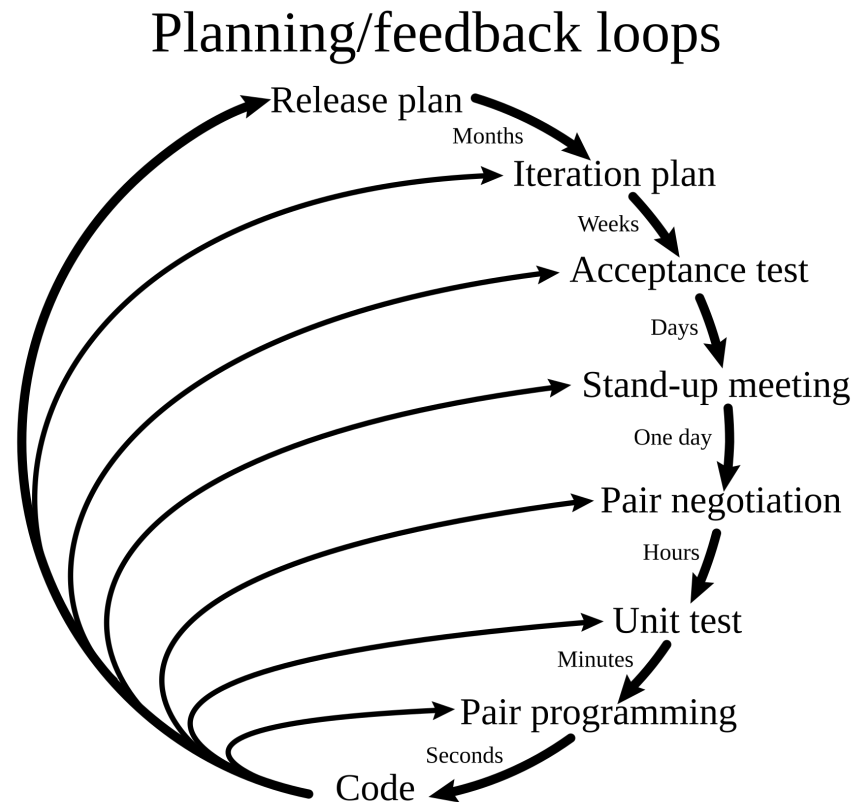


- Die Rechte an geschützten Marken liegen bei den jeweiligen Markeninhabern
- Die Rechte an referenzierte Literatur, Folien, Notizen und sonstigen Materialien liegen bei den jeweiligen Autoren
- Diese Vorlesung benutzt Materialien von Prof. Dr. Spillner (HSB in Ruhestand)
- Alle Rechte an den Materialien zu dieser Veranstaltung liegen bei ihrem Autor, Prof. Dr.-Ing. Jasminka Matevska
- Jede Form der teilweisen oder vollständigen Weitergabe, Speicherung auf Servern oder Nutzung in Lehrveranstaltungen, die nicht von dem Autor selbst durchgeführt werden, erfordert seine schriftliche Zustimmung. Eine schriftliche Zustimmung ist darüber hinaus für jede kommerzielle Nutzung erforderlich
- Für inhaltliche Fehler kann keine Haftung übernommen werden

- Anforderungs-Engineering
- **Scrum**
- Modellierung der Struktur mit UML (inverted Classroom)
- Modellierung der Dynamik mit UML (inverted Classroom)
- Clean Code
- Testen
- Entwurfsmuster
- Softwarearchitektur für Entwickler
- Clean Coder
- Retrospektive



- Entwickelt von Kent Beck
- 1996 bei Chrysler im Projekt
- Eine leichtgewichtige Methode, die auf Entwickler:innen abgezielt ist
- Soll durch Praktiken und Prinzipien und kürzere Feedback-Schleifen höhere Qualität erzielen



Quelle: https://en.wikipedia.org/wiki/Extreme_programming

Aktivitäten:

- Coding
- Testing
- Listening
- Designing

Werte:

- Communication
- Simplicity
- Feedback
- Courage
- Respect

Prinzipien:

- Feedback
- Assuming simplicity
- Embracing change

Quelle: https://en.wikipedia.org/wiki/Extreme_programming

- 2001 trafen sich 17 Softwareentwickler in Utah, um über leichtgewichtige Entwicklungsmethoden zu Diskutieren
- Dazu gehörten:
 - Kent Beck (eXtreme Programming, TDD)
 - Ward Cunningham (eXtreme Programming, Wiki)
 - Dave Thomas (Pragmatic Programming, Ruby)
 - Jeff Sutherland (Scrum)
 - Ken Schwaber (Scrum)
 - Jim Highsmith (Adaptive Software Development)
 - Alistair Cockburn (Crystal Method, Heart of Agile)
 - Robert "Uncle Bob" C. Martin (Clean Code, SOLID)
 - Mike Beedle (Scrum)
 - Arie van Bennekum
 - Martin Fowler (Refactoring, OOAD, UML)
 - James Grenning
 - Andrew Hunt (Pragmatic Programming, Ruby)
 - Ron Jeffries (eXtreme Programming)
 - Jon Kern
 - Brian Marick (Ruby, TDD)
 - Steve Mellor (OOA)

<https://agilemanifesto.org>

Werte:

- Individuen and Interaktionen über Prozesse und Werkzeuge
- Funktionierende Software über verständliche Dokumentation
- Zusammenarbeit mit dem Kunden über Vertragsverhandlungen
- Auf Veränderung reagieren über einen Plan verfolgen

SCRUM

Quelle der folgenden Seiten: <https://scrumguides.org/scrum-guide.html>

SCRUM
GUIDES

[Web Version](#)
[PDF Versions](#)
[History](#)
[FAQ](#)



Read the official Scrum Guide online

[Read Online >](#)



Download the official Scrum Guide
PDF in English

[Download \(English\) >](#)



Download the official Scrum Guide
in over 30 different languages

[Select language & Download >](#)

What is Scrum?

Scrum is a framework for developing and sustaining complex products. This Guide contains the definition of Scrum. This definition consists of Scrum's accountabilities, events, artifacts, and the rules that bind them together. Ken Schwaber and Jeff Sutherland developed Scrum; the Scrum Guide is written and provided by them. Together, they stand behind the Scrum Guide.

Share Your Ideas!

If you have ideas for improving the website or the Scrum Guide itself, please share them! You may do so by sending a message to our support e-mail.

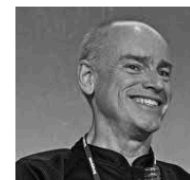
[Share an idea](#)

About the creators of Scrum



Meet Jeff Sutherland

Jeff is the co-creator of Scrum and a leading expert on how the framework has evolved to meet the needs of today's business...

[Read Jeff's Bio](#)


Meet Ken Schwaber

Ken Schwaber co-developed the Scrum process with Jeff Sutherland in the early 1990s to help organizations...

[Read Ken's Bio](#)

Quelle: <https://scrumguides.org/index.html>

- Definition
- Theorie
- Werte
- Team
- Ereignisse
- Artefakte

Scrum: Definition

- Scrum ist ein leichtgewichtiges Rahmenwerk, welches Menschen, Teams und Organisationen hilft, Wert durch adaptive Lösungen für komplexe Probleme zu generieren - *Scrum Guide 11.2020*

In a nutshell, Scrum requires a Scrum Master to foster an environment where:

1. A Product Owner orders the work for a complex problem into a Product Backlog.
2. The Scrum Team turns a selection of the work into an Increment of value during a Sprint.
3. The Scrum Team and its stakeholders inspect the results and adjust for the next Sprint.
4. *Repeat*

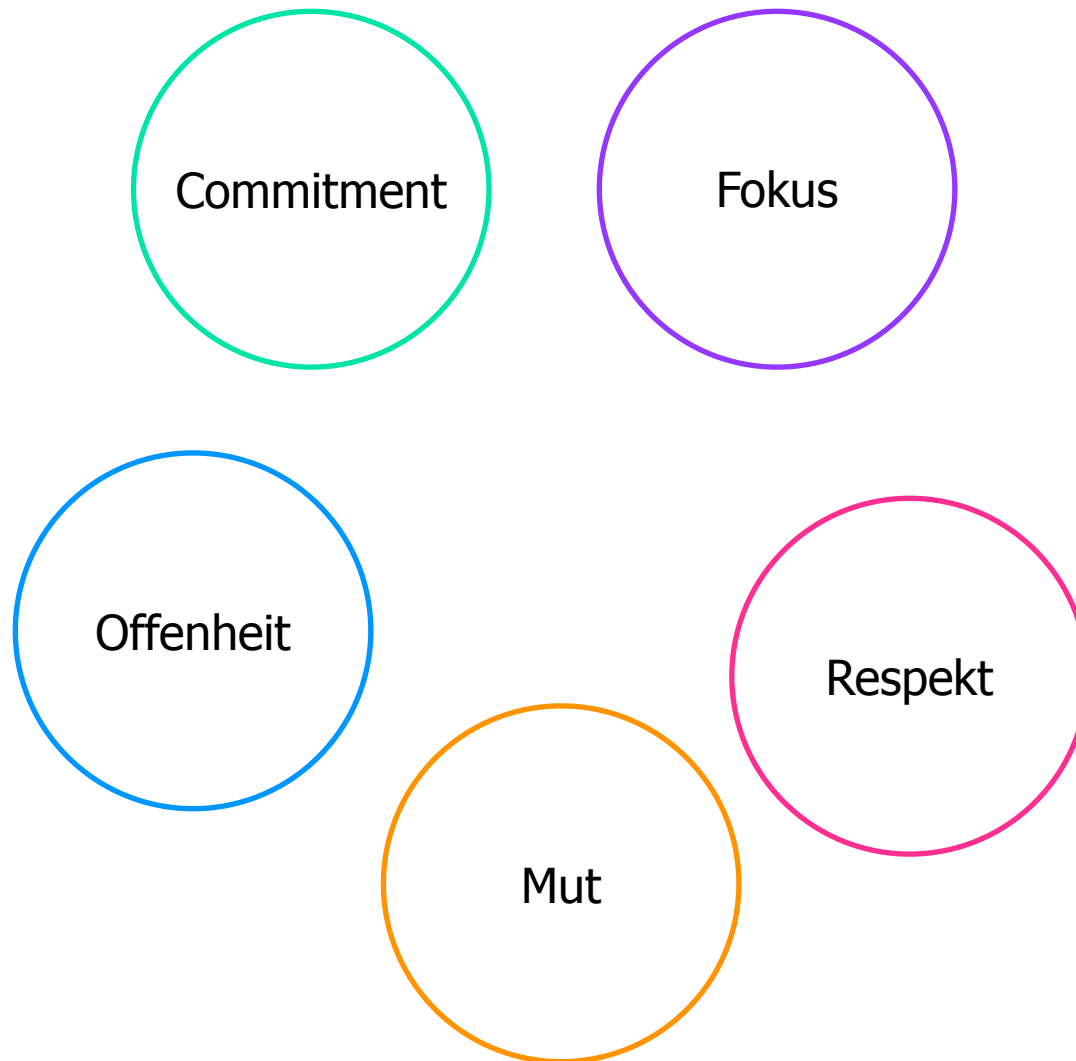


<https://de.freepik.com/fotos/der-gute-hirte>

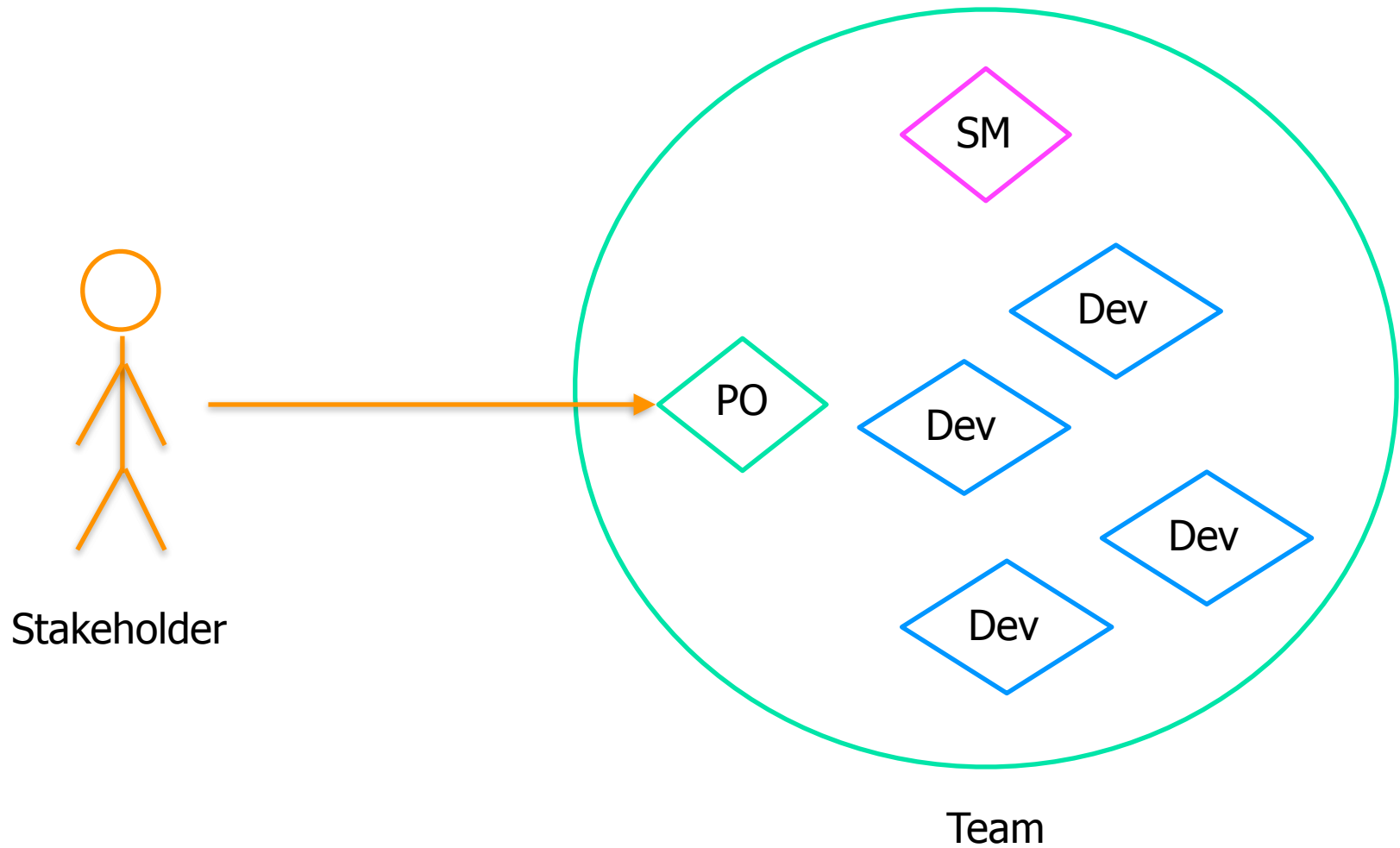
Scrum: Theorie



Scrum: Werte



Scrum: Team



Verantwortlich für:

- einen Plan für den Sprint zu erstellen, das Sprint Backlog
- Qualität durch die Einhaltung einer Definition of Done einzubringen
- täglich ihren Plan zur Erreichung des Sprint-Ziels anzupassen und
- sich wechselseitig als Expert:innen zur Verantwortung zu ziehen.

Verantwortlich für:

- das Produkt-Ziel zu entwickeln und explizit zu kommunizieren
- die Product-Backlog-Einträge zu erstellen und klar zu kommunizieren
- die Reihenfolge der Product-Backlog-Einträge festzulegen und
- sicherzustellen, dass das Product Backlog transparent, sichtbar und verstanden ist.

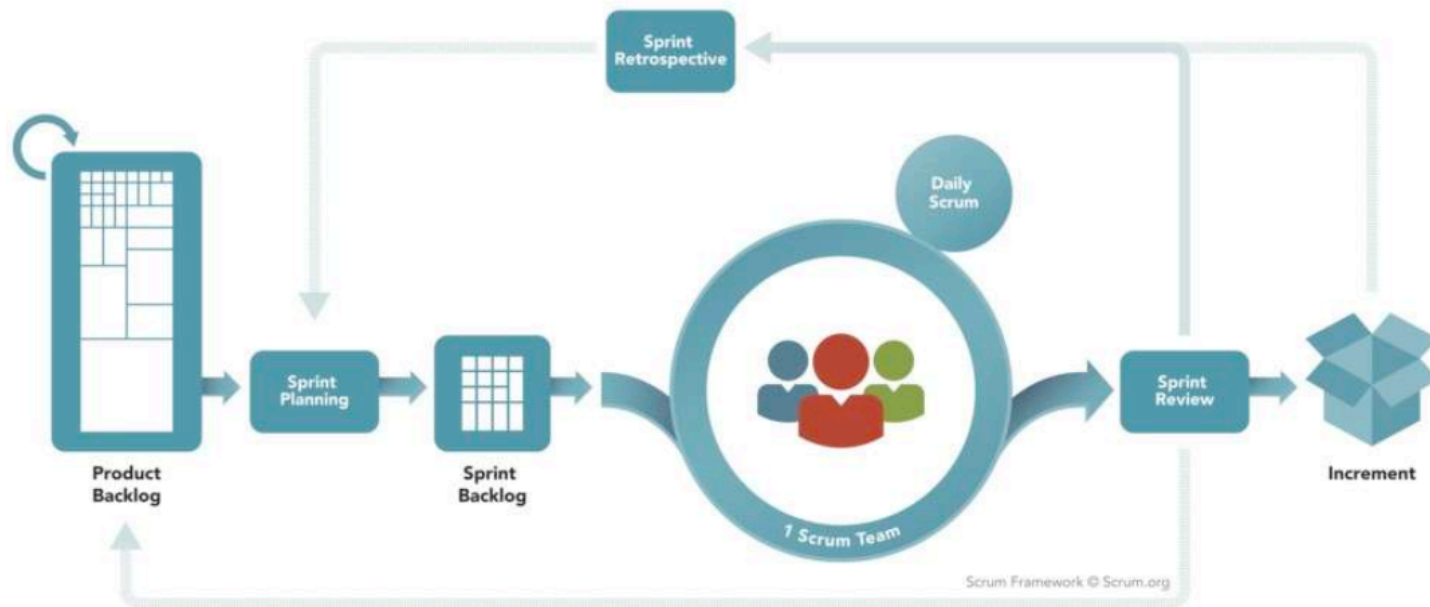
Verantwortlich für:

- Die Effektivität des Teams
- Die Gesamtorganisation
- Das die Methode (das Scrum Framework) korrekt durchgezogen wird
 - oft Moderator

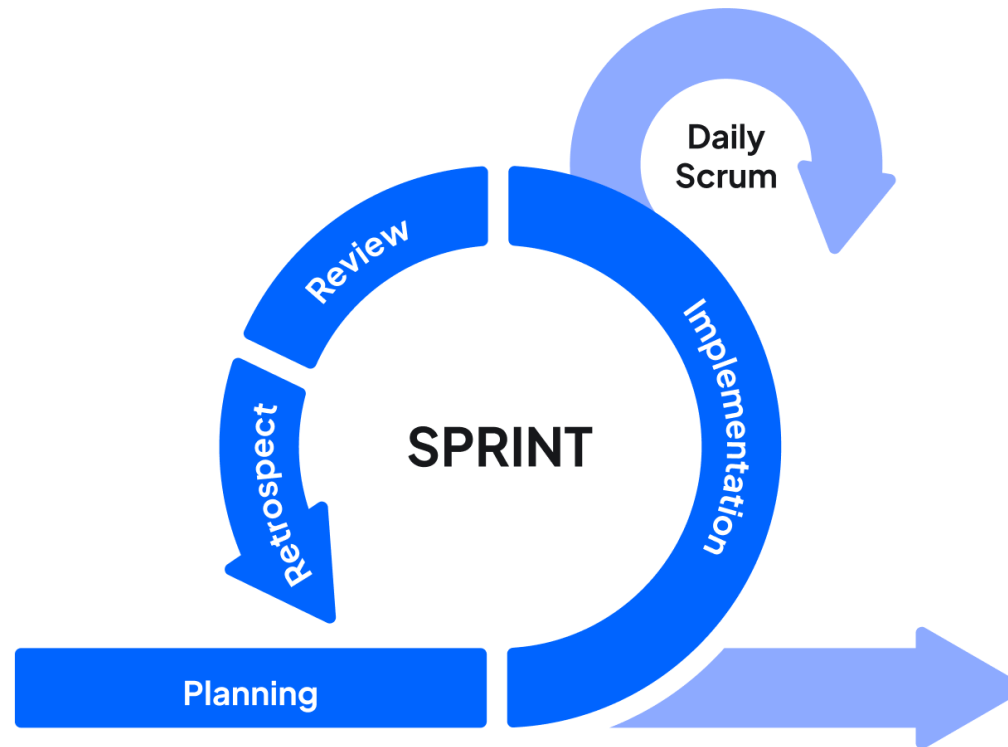
Zusätzlich:

- Fachlich und technisch neutraler Beobachter des Teams
 - z. B. Zwischenmenschliche Probleme früh erkennen und gegen wirken

Scrum: Ereignisse



- Sprint
- Planning
- Daily
- Review
- Retrospektive



Quelle: <https://ones.com/blog/sprint-review>

Während eines Sprints

- Keine Änderungen werden gemacht, die das Sprintziel in Gefahr bringen
- Die Qualität nimmt nicht ab
- Das Product Backlog ist aufbereitet wie benötigt
- Der Umfang kann klargestellt oder neu vereinbart werden, wenn neue Erkenntnisse erlangt werden.

- Thema 1: Warum ist der Sprint wertvoll?
- Thema 2: Was kann im Springt abgeschlossen (Done) werden?
- Thema 3: Wie wird die ausgewählte Arbeit erledigt?

Dauer: ca 15 Min. täglich

Zweck:

- Sprint-Ziel überprüfen
- Synchronisation des Stands der Aufgaben
- Identifikation von Hindernissen
- Fördert schnelle Entscheidungsfindung

Zweck:

- Das Ergebnis des Sprints überprüfen und künftige Anpassungen festlegen
- Vorstellung der Ergebnisse den wichtigsten Stakeholder:innen
- Fortschritte in Richtung Produkt-Ziel mit den Stakeholder:innen diskutieren

Dauer:

- Bei einmonatigen Sprints max. drei Stunden
- Bei kürzeren Sprints in der Regel kürzer

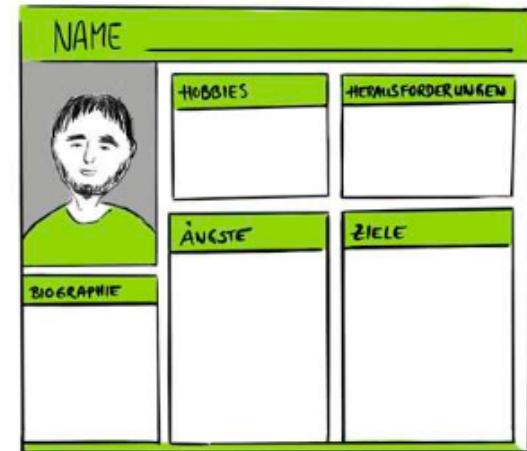
Zweck:

- Das Scrum-Team Überprüfung des letzten Sprints in Bezug auf:
 - Individuen
 - Interaktionen
 - Prozesse
 - Werkzeuge
 - Definition of Done

- Start/Stop/Continue
- 4L: Liked/Learned/Lacked/Longed for
- Starfish: Start/Stop/Keep/More of / Less of
- ...

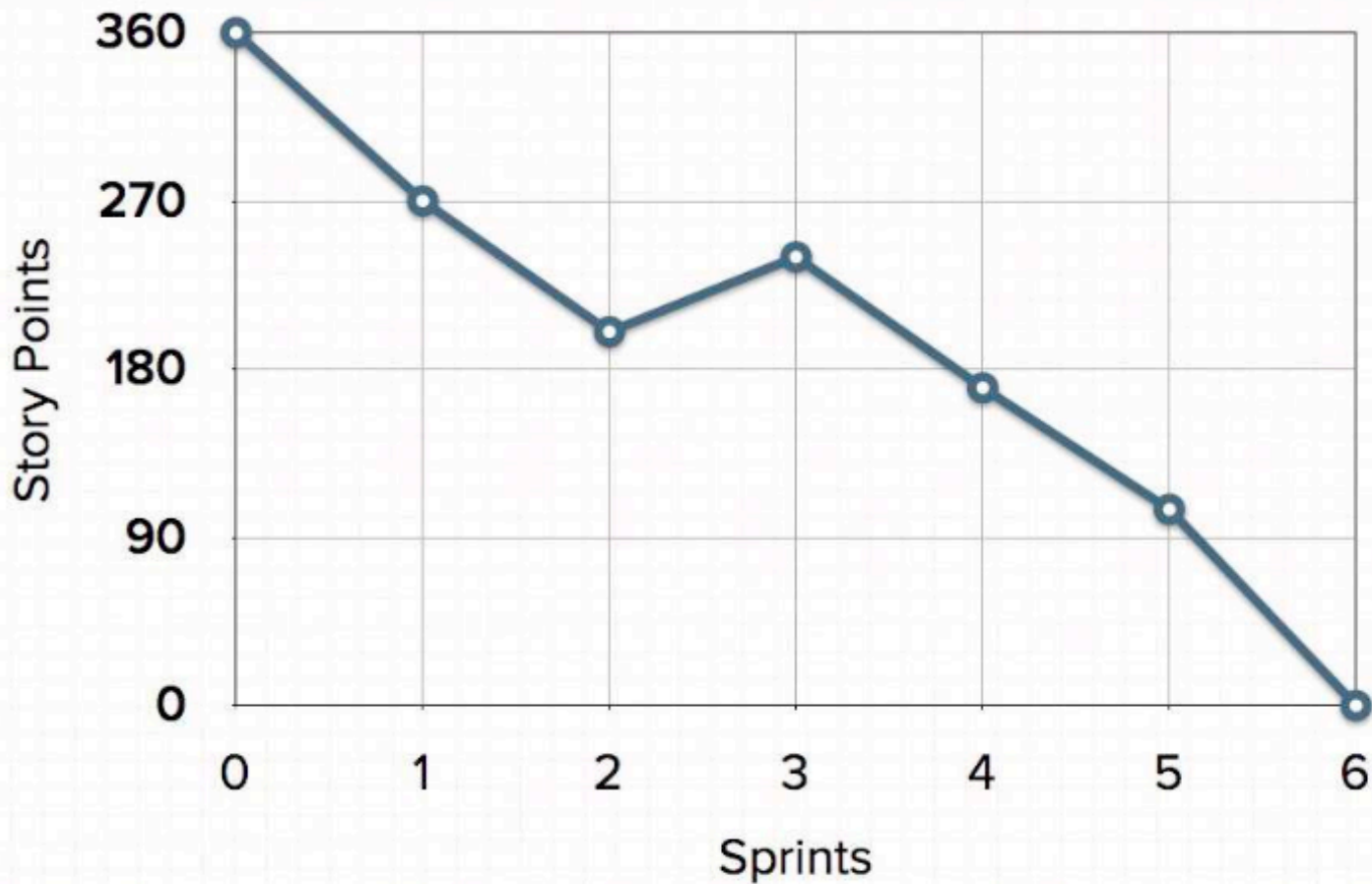
Personas

- Stereotypische Darstellung eine Klasse von Benutzern
- Soll helfen mehr über die Zielgruppen zu erfahren und greifbarer zu machen
- Mögliche Eigenschaften:
 - Name, Alter und Geschlecht
 - Beruf, Funktion, Verantwortlichkeiten
 - Fachliche Ausbildung, Wissen und Fähigkeiten
 - Charakterzüge
 - Verhaltensmuster und Vorgehensweisen
 - Ziele, Werte, Ängste, Sehnsüchte und Vorlieben
 - Produktkenntnisse
 - Allgemeine Computerkenntnisse
 - Erwartungen an die neue Lösung
- Dient als "Wer" als Akteure in den User Stories verwendet



[Projektmagazin.de]

Scrum: Burndown Chart



[<http://www.mountaingoatsoftware.com/agile/scrum/scrum-tools/release-burndown>]

- Fortschritt in Richtung des Ziels überwachen
- Zu jedem Zeitpunkt kann die zur Zielerreichung verbleibende Arbeit summiert werden
 - Wir zumindest bei jedem Sprint Review ermittelt
- Aus Vergleichen der restlichen Arbeit mit bereits abgeschlossenen Sprints kann die Geschwindigkeit und somit der Fertigstellungstermin beurteilt werden
- Einsatz von Trend-, Burndown-, Burnup- etc. Techniken
- Verantwortlich: Der Scrum Master

Scrum: Artefakte

- Product Backlog
- Sprint Backlog
- Inkrement

Besteht aus:

- geordnete Liste der Dinge, die zur Produktverbesserung benötigt werden
- Nur dort stehen alle Aufgaben, die mit dem Kunden vereinbart von Scrum Team erledigt werden

Beispiel-Template für
ein Backlog-Item

UserStory
Als möchte ich um
Akzeptanzkriterien
• Punkt 1 • Punkt 2 Welche Rechte dürfen die Aktion durchführen? z.B <Ausgabe ändern, Fibu Daten ändern, Mandanten Administration, Systemadministration
Infos

Besteht aus:

- dem Sprint-Ziel
- ausgewählte Product-Backlog-Einträge (Was)
- einem umsetzbaren Plan für die Lieferung des Inkrements (Wie)

Ist ein:

- konkreter Schritt in Richtung Produkt-Ziel
- Mehrwert, der auch verwendbar ist

Commitment: Definition of Done

- Die Definition of Done ist eine formale Beschreibung des Zustands des Inkrements, die erfüllt sein müssen, damit die Aufgabe als erfüllt gilt.

- Timebox
 - Alles in einem Sprint bekommt ein Zeitfenster. Dieses soll möglichst eingehalten werden (um z. B. zu ausartende Meetings zu vermeiden)
- Inkrement
 - meist Release bezeichnet, da es oft aus mehreren Inkrementen besteht
- Features
 - Sammlung von Anforderungen, die zusammen einen Mehrwert liefern

Agile Schätzmethoden

- Relative Schätzung (Story Points)
- Story A ist komplexer als B
- Story C ist ähnlich komplex wie B
- Schätzen in Größenordnungen (Schätzskalen)
 - Vereinfachte Fibonacci-Reihe (z.B. 1, 2, 3, 5, 8, 13, 20, 40, 100)
 - T-Shirt Größen: XXS, XS, S, M, L, XL, XXL und XXXL

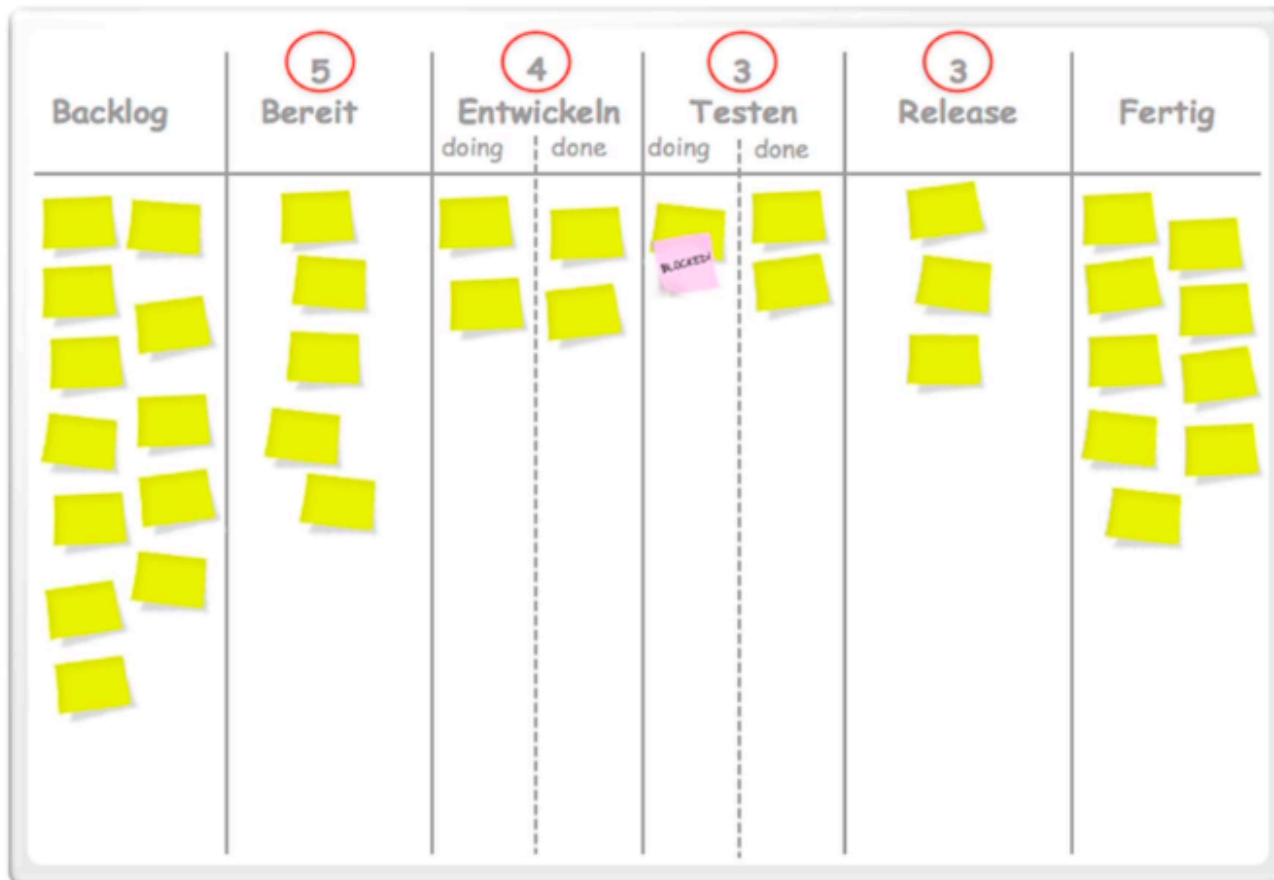
- Vorbereitung:
 - Alle zu schätzenden Stories liegen auf einzelnen Karten verdeckt vor
 - Das Scrum-Team ist komplett anwesend
- Schätzen:
 - Ein Teammitglied nimmt die erste Story, liest sie laut vor und hängt sie an die Wand
 - Der zweite nimmt die nächste Karte, liest sie laut vor und hängt sie an die Wand:
 - Gleiche Komplexität -> unter der ersten Karte
 - Größere Komplexität -> vor der ersten Karte (rechts)
 - kleinere Komplexität -> hinter der ersten Karte
 - Ab der dritten Karte gibt es folgende Möglichkeiten:
 - Ausspielen der nächsten Karte wie oben beschrieben
 - Umhängen einer Karte mit kurzer Begründung
 - Aussetzen

- Spielende: alle Karten an der Wand und alle Spieler aussetzen
- Abrechnung
 - Die äußerste linke Spalte enthält den Wert 1
 - Die weiteren Spalten erhalten von links nach rechts die Werte 2, 3, 5, 8, 13, 20, 40, 100
 - Wenn mehr als 9 Spalten vorliegen, fasst die Gruppe solange benachbarte Spalten mit eng liegenden Komplexitätswerten zusammen bis maximal 9 Spalten vorhanden sind
- Die relativen Story Points sind einfacher zu schätzen
- Fragen:
 - Wie lange dauert die Bearbeitung wirklich?
 - Wie viele Punkte sind „ein Tag“ Aufwand?
 - Wie viel kann in einer Iteration erledigen?

- Planungspoker - relative Komplexität schätzen
- Vorbereitung: jedes Teammitglied hat ein Kartensatz
- Kalibrieren: kleinste Anforderung wählen und mit 1 Komplexitätspunkt bewerten (oder eine typische z.B. mit 5)
- Schätzen:
 - Vorstellen der Anforderung
 - Jeder wählt eine Schätzkarte
 - Karten werden aufgedeckt
 - Diskussion (über max. bzw. min.)
 - ggf. Wiederholung der Schätzung
 - Wenn keine Einigung, entscheiden was zu tun (mitteln, max bzw. min raus, vertragen...)

- Epic
 - Feature-Beschreibung auf höchster Abstraktionsstufe
 - Meist zu groß für einen Sprint
 - Wird im Team in mehrere kleinere User Stories zerlegt
- User Story
 - Beschreibt ein mögliches Szenario
 - Auf dieser Basis schätzt das Team den Realisierungsaufwand ein
 - kein prinzipieller Unterschied zu klassischen Use-Case basierten Methoden
- Task
 - Entwickler:in und Tester:in zerlegen die User Stories in Aufgaben

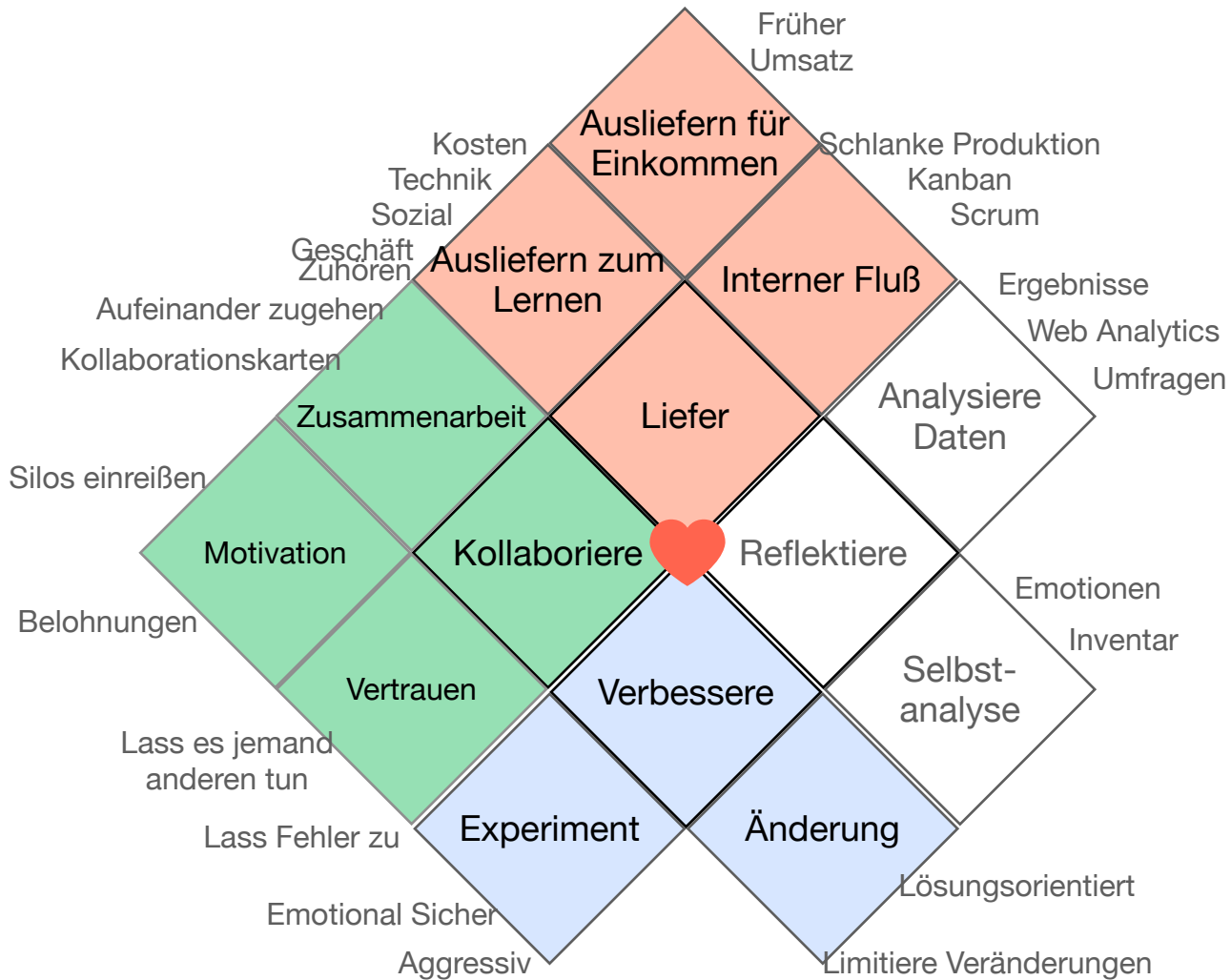
Agiles Kanban Board



[it-agile.de]

Heart of Agile





- Das gesamte Team und Kunde sollen an einem Ort arbeiten, damit eine gute Kommunikation gewährleistet werden kann
- Die Größe eines Entwicklungsteams sollten nicht größer als 7 sein
- Personen sein, sonst steigt der Kommunikationsaufwand überproportional
- Die Kommunikation über Teamgrenzen hinweg ist kostspielig und manchmal langwierig, jedoch notwendig bei großen Organisationen
- Skalierbarkeit (Koordination von mehreren Scrum-Teams) schwierig
 - Scrum of Scrums
 - Large Scale Scrum – LeSS
 - Scaled Agile - SAFe
- Die meisten machen Scrum-but (Wir machen Scrum, aber...)
- Scrum in einer "klassischen" Organisation
- Sprint von außen abschirmen
- Cargo Culture

- Was besagt das Manifesto for Agile Software Development?
- Grundbegriffe des Agilen Projektmanagements sind: ...
- Erklären Sie die Begriffe: Product Backlog, User Story, Daily/Stand-Up
- Welche Rollen gibt es im Scrum?
- Agile Schätzungsmethoden
- Was ist das Ergebnis eines Sprint-Reviews?
- Was ist DoD?
- Eine Retrospektive ist ... Ihre Ziele sind ...
- Was ist agil-typisch bei der Kommunikation im Projekt?
- Was bedeutet “Customer-on-site”?
- Welche Einschränkungen / Herausforderungen gibt es bei Scrum?

We follow these principles:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity--the art of maximizing the amount of work not done--is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Quelle: <https://agilemanifesto.org/principles.html>

- **Primär: Materialien im AULIS!**
- I. Sommerville - Software Engineering, 2012
- M. Broy, M. Kuhrmann. Projektorganisation und Management im Software Engineering, 2013
- Project Management Institute (PMI) – A Guide to the Project Management Body of Knowledge (PMBOK Guide) - 5th Edition, 2013
- K. Schwaber, J. Sutherland. The Scrum Guide, 2020
- INCOSE Systems Engineering Handbook, 2015
- ISO/IEC/IEEE 16326:2009 - Systems and software engineering -- Life cycle processes -- Projectmanagement
- C. Rupp, S. Queins, die SOPHISTen. UML 2 glasklar: Praxiswissen für die UML-Modellierung, 2012
- UML 2.5 - OMG Unified Modeling Language Version 2.5
- E. Gamma, R. Helm, R. E. Johnson. Design Patterns. Elements of Reusable Object-Oriented Software, 1996
- P. Clements. Documenting Software Architectures: Views and Beyond, 2003
- L. Bass, P. Clements, R. Kazman. Software Architecture in Practice, 2012
- F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad – A System of Patterns, Pattern-Oriented Architecture, 2000
- T. Erl. Service-Oriented Architecture: Analysis and Design for Services and Microservices, 2016
- R. Reussner, W. Hasselbring (Hrsg.). Handbuch der Software-Architektur, 2008
- J. Matevska. Rekonfiguration komponentenbasierter Softwaresysteme zur Laufzeit, 2010